INPT

Institut National des Postes et Télécommunications

Filière Innovation & AMOA

Année universitaire 2026–2027

BIG DATA ANALYTICS

Plateforme d'analyse de graphe de connaissances propulsée par l'IA

Évaluation comparative de l'analytique de graphe structurée (Cypher, Graph Data Science) et du GraphRAG sur un graphe construit automatiquement à partir de sources hétérogènes

HDFS · Apache Spark · LLM Gemini · Neo4j · GDS · GraphRAG

RÉALISÉ PAR

AKHI Salma

BENTATOU Rania

ENCADRÉ PAR

M. EL GHAZI Hamid

Table des matières

1. Introduction 3
1.1 Questions de recherche 3
2. Architecture de la plateforme 3
3. Phase 1 — Collecte et ingestion dans HDFS 4
4. Phase 2 — Extraction d'entités et de relations par LLM 6
5. Phase 3 — Transformations Spark et résolution d'entités 6
6. Chargement dans Neo4j 8
7. Phase 4 — Graph Analytics (Cypher et Graph Data Science) 10
8. Phase 5 — GraphRAG et Text2Cypher 12
8.1 Pipeline GraphRAG 12
8.2 Text2Cypher 12
9. Phase 6 — Évaluation 14
9.1 Qualité de l'extraction 14
9.2 Benchmark comparatif 15
9.3 Analyse des erreurs 16
10. Phase 7 — Application, API, monitoring et contrôle d'accès 17
11. Réponses aux questions de recherche 21
12. Éthique, conformité et gouvernance des données 22
13. Limites et perspectives 22
14. Conclusion 22
Annexe A — Exécution du projet 23
Annexe B — Organisation du code 23

1. Introduction

Les organisations disposent d'informations dispersées dans des sources hétérogènes : pages encyclopédiques, flux d'actualités, documents internes, bases structurées. Les relations entre entités (qui a fondé quoi, qui a racheté qui, qui investit dans qui) y sont exprimées en langage naturel et ne sont donc pas directement interrogeables.

Ce projet construit une **plateforme d'analyse de graphe de connaissances** qui collecte ces sources, les stocke dans un environnement Big Data (HDFS), utilise un **grand modèle de langage (LLM)** pour en extraire entités et relations, nettoie et fusionne les résultats avec **Apache Spark**, puis charge un graphe unique dans **Neo4j**. Ce graphe est ensuite exploité par deux approches analytiques que l'on compare :

- **Graph Analytics** : requêtes Cypher directes et algorithmes de Neo4j Graph Data Science (GDS) ;
- **GraphRAG** : récupération d'un sous-graphe pertinent combinée à un LLM qui répond en langage naturel.

Une troisième approche, **Text2Cypher** (le LLM traduit la question en requête Cypher), a été ajoutée comme point de comparaison intermédiaire. Le domaine choisi est l'**écosystème des entreprises technologiques** (fondateurs, acquisitions, investissements, produits), riche en questions à plusieurs sauts (multi-hop).

1.1 Questions de recherche

1. Avec quelle précision un LLM extrait-il entités et relations de documents hétérogènes ?
2. Quel est l'impact de la résolution d'entités sur la qualité du graphe obtenu ?
3. Comment GraphRAG se compare-t-il aux requêtes Cypher directes pour les questions multi-hop et sémantiques ?
4. Quels compromis existent entre qualité des réponses, complétude, latence et coût d'inférence ?
5. Comment la provenance des sources améliore-t-elle la fiabilité et l'explicabilité des réponses ?

2. Architecture de la plateforme

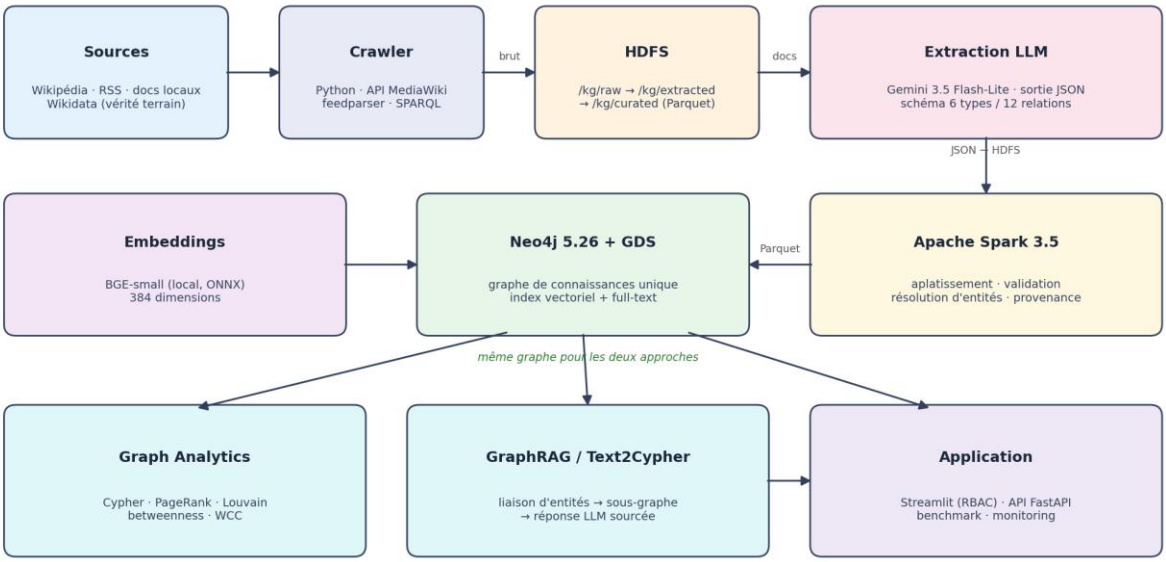


Figure 1 – Architecture globale : un seul graphe de connaissances alimente les deux approches analytiques

Le principe de conception central est de maintenir **un seul graphe** servant aux deux approches, ce qui rend la comparaison équitable : Cypher et GraphRAG interrogent exactement les mêmes données. L'ensemble est déployé en preuve de concept avec **Docker Compose** :

Tableau 1 – Services de la plateforme

Service (conteneur)	Technologie	Rôle	Accès
kg-namenode / kg-datanode	Hadoop HDFS 3.2	Stockage brut, extrait et curé (JSONL, Parquet)	localhost:9870
kg-app	Python 3.11, PySpark 3.5.3	Crawler, extraction LLM, jobs Spark, interface Streamlit	localhost:8501
kg-neo4j	Neo4j 5.26 + GDS 2.13 + APOC	Graphe, index vectoriel et full-text, algorithmes	localhost:7475
kg-api	FastAPI	API REST sécurisée par clé (GraphRAG, Cypher)	localhost:8000/docs
(externe)	Gemini 3.5 Flash-Lite	Extraction, liaison d'entités, génération	API Google

Choix techniques notables :

- **Spark en mode local[*]** dans le conteneur applicatif, lisant et écrivant sur HDFS (hdfs://namenode:9000). Le même code s'exécute sur un cluster en changeant seulement la variable SPARK_MASTER.
- **Schéma de graphe partagé** (common/config.py) : 6 types d'entités et 12 types de relations, utilisés à la fois par l'extraction, le chargement, Text2Cypher et GraphRAG.
- **Embeddings locaux** (BAAI/bge-small-en-v1.5 via ONNX) : le quota gratuit de l'API d'embeddings Gemini (~100 textes/min) était insuffisant pour 1 473 entités ; le modèle local supprime ce coût et cette latence.
- **Cache disque des appels LLM** : relancer le pipeline ne refait pas les appels déjà payés.

3. Phase 1 — Collecte et ingestion dans HDFS

Le crawler (pipeline/01_crawl.py) collecte quatre types de sources de nature différente :

Tableau 2 – Sources collectées

Source	Format	Volume	Usage
Wikipédia (40 entreprises)	Texte non structuré (API MediaWiki JSON)	40 articles	Corpus principal
Flux RSS (TechCrunch, The Verge, Ars Technica)	Semi-structuré (XML)	18 articles filtrés	Actualités récentes
Document local (note interne)	Markdown	1 document	Documents d'entreprise
Wikidata (SPARQL)	Structuré (JSON)	245 faits	Vérité terrain pour l'évaluation

Au total 59 documents (375 054 caractères) sont écrits dans HDFS sous /kg/raw/<source>/. Les faits Wikidata ne sont pas envoyés au LLM : ils servent uniquement de référence indépendante pour mesurer la qualité de l'extraction. Le crawler respecte la politique d'utilisation de Wikimedia (User-Agent identifié, pauses et réessais sur HTTP 429).

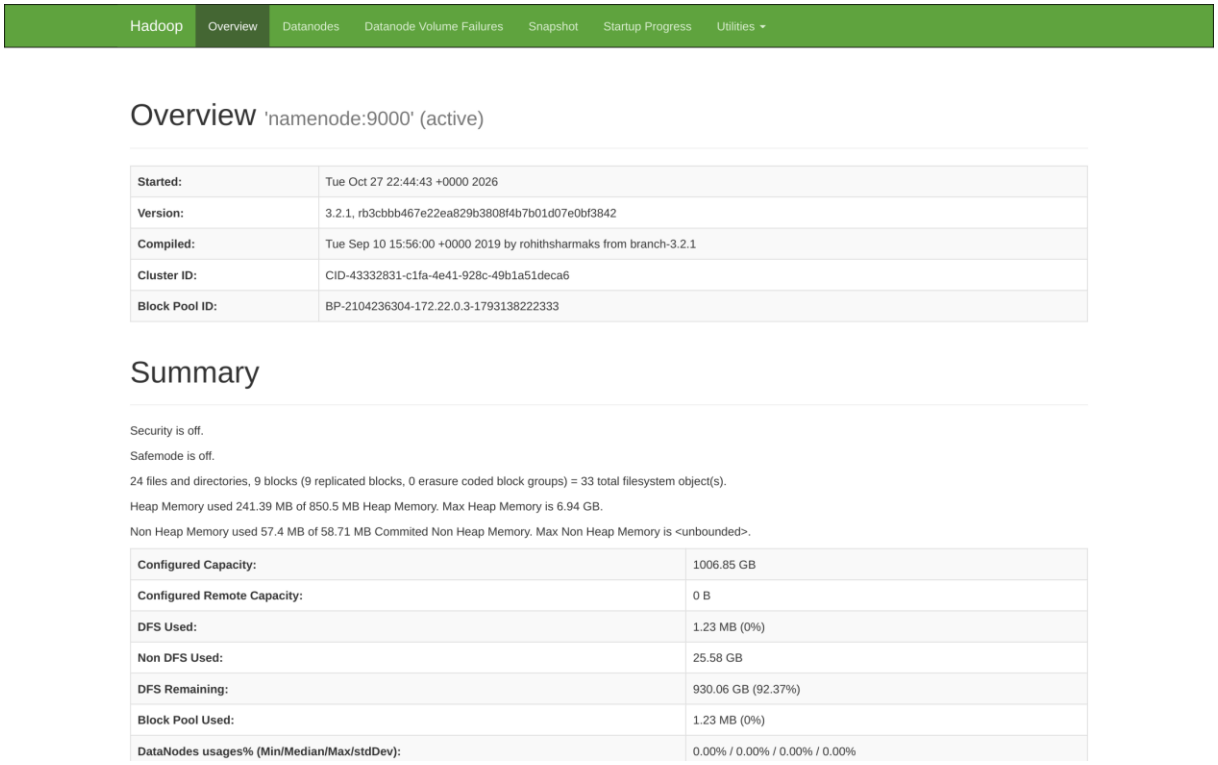


Figure 2 – Interface du NameNode HDFS : cluster opérationnel

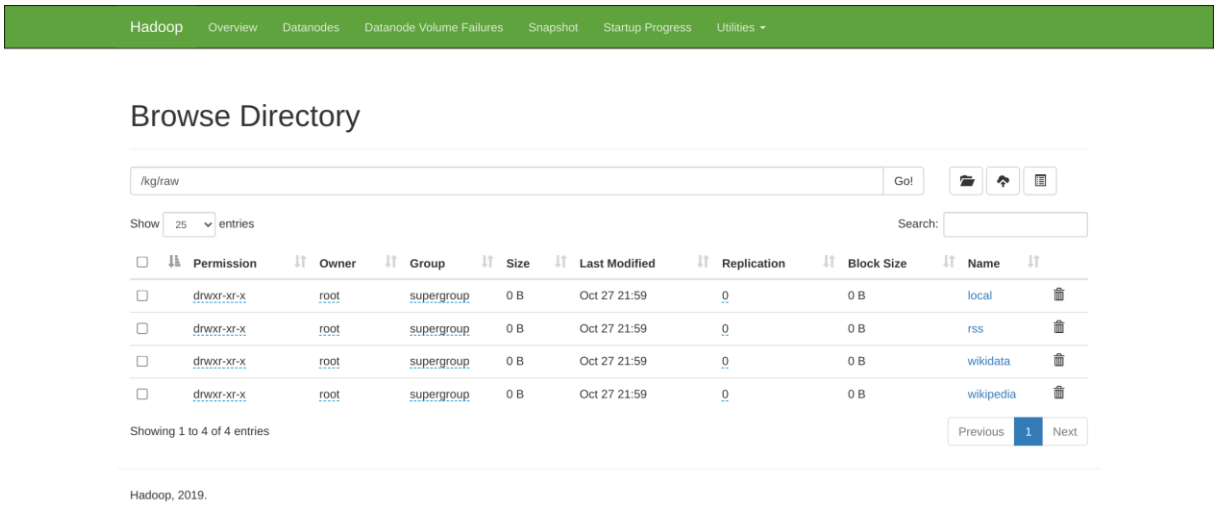


Figure 3 – Zone brute HDFS /kg/raw : une arborescence par source

4. Phase 2 — Extraction d'entités et de relations par LLM

Chaque document est découpé en morceaux d'environ 4 500 caractères. Pour chaque morceau, Gemini 3.5 Flash-Lite reçoit le schéma du graphe et doit renvoyer un **JSON contraint par un schéma** (responseSchema) : liste d'entités (nom, type, description) et de relations (source, relation, cible, **phrase de preuve**). La phrase de preuve est conservée jusque dans Neo4j : c'est la base de la provenance.

Tableau 3 – Types de relations du schéma (types d'entités : Company, Person, Product, Technology, Location, Organization)

Relation	Sens	Signification
FOUNDED	(Person|Company) → (Company|Organization|Product)	source founded or co-founded target
CEO_OF	(Person) → (Company|Organization)	source is or was CEO of target
WORKS_FOR	(Person) → (Company|Organization)	source works or worked for target (not CEO)
ACQUIRED	(Company) → (Company|Product)	source acquired or bought target
OWNS	(Company|Person) → (Company|Product)	source owns target (without explicit acquisition)
SUBSIDIARY_OF	(Company) → (Company)	source is a subsidiary / division of target
INVESTED_IN	(Company|Person|Organization) → (Company)	source invested in target
DEVELOPS	(Company|Person|Organization) → (Product|Technology)	source develops / makes target
HEADQUARTERED_IN	(Company|Organization) → (Location)	source has its headquarters in target
PARTNERS_WITH	(Company|Organization) → (Company|Organization)	partnership or collaboration
COMPETES_WITH	(Company) → (Company)	source competes with target
RELATED_TO	(*) → (*)	any other meaningful relation (use rarely)

Extrait des règles du prompt :

```
- Only extract facts explicitly stated in the text. Do not use outside knowledge.
- Use the most complete common name for each entity (e.g. "Mark Zuckerberg", not "Zuckerberg").
- Every relation's source and target must also appear in the entities list.
- "evidence" is the short sentence (max 30 words) from the text that supports the relation.
```

Tableau 4 – Statistiques de l'extraction

Indicateur	Valeur
Documents / morceaux traités	59 / 138
Appels en erreur	0
Entités extraites (mentions brutes)	2120
Relations extraites (brutes)	1470
Tokens entrée / sortie	149 394 / 179 892
Coût estimé (tarif payant)	0.09 $
Durée (4 appels parallèles)	7.4 min

5. Phase 3 — Transformations Spark et résolution d'entités

Le job Spark (pipeline/03_spark_transform.py) lit les extractions JSON depuis HDFS et réalise :

- l'**aplatissement** des listes imbriquées d'entités et de relations (explode) ;
- la **résolution d'entités** en trois étapes : (1) normalisation (casse, accents, ponctuation, suffixes juridiques « Inc. », « Corporation », « Platforms »…), (2) harmonisation du type (type majoritaire), (3) rapprochement flou (rapidfuzz, seuil 93) et règle des noms de famille pour les personnes (« Zuckerberg » → « Mark Zuckerberg ») ; la table de correspondance est diffusée (broadcast) aux exécuteurs ;
- la **validation** des relations (type autorisé, extrémités connues, pas de boucle) ;

- l'**agrégation de la provenance** : poids = nombre de mentions, liste des documents sources, phrases de preuve ;
- l'écriture du résultat en **Parquet** dans /kg/curated/.

Tableau 5 – Effet de chaque étape de la résolution d'entités

Étape	Nombre	Commentaire
Mentions d'entités	2115	sorties brutes du LLM
Formes de surface distinctes	1554	couples (nom, type) distincts
Après normalisation	1488	−66 doublons
Conflits de type corrigés	32	même entité typée différemment
Fusions floues	15	variantes orthographiques, noms de famille
Entités résolues	1473	réduction totale : 5.2 %
Relations brutes → curées	1470 → 1324	10 invalides rejetées, doublons agrégés

Tableau 6 – Exemples de fusions réalisées

Entité canonique	Type	Alias fusionnés
Broadcom	Company	Broadcom, Broadcom Corporation, Broadcom Inc., Broadcom Limited
Nvidia	Company	NVIDIA, Nvidia, Nvidia Corporation
Google	Company	Google, Google LLC, Google+
GitHub	Company	GitHub, GitHub, Inc, GitHub, Inc.
Jeff Bezos	Person	Bezos, Jeff Bezos
SpaceXAI	Company	SpaceXAI, SpaceXAI LLC
Mike Krieger	Person	Krieger, Mike Krieger
WhatsApp	Product	WhatsApp, WhatsApp Inc.
X Corp.	Company	X Corp, X Corp.
Netflix	Company	Netflix, Netflix, Inc.

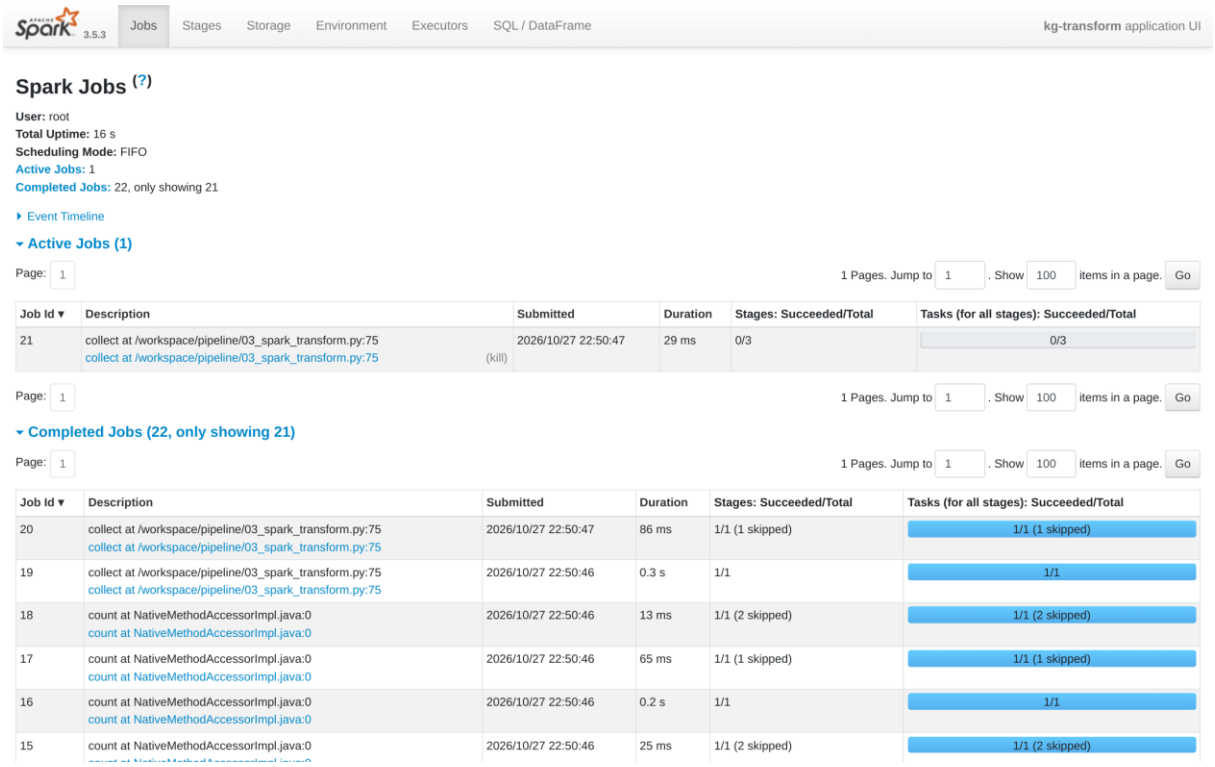


Figure 4 – Interface Spark pendant l'exécution du job kg-transform

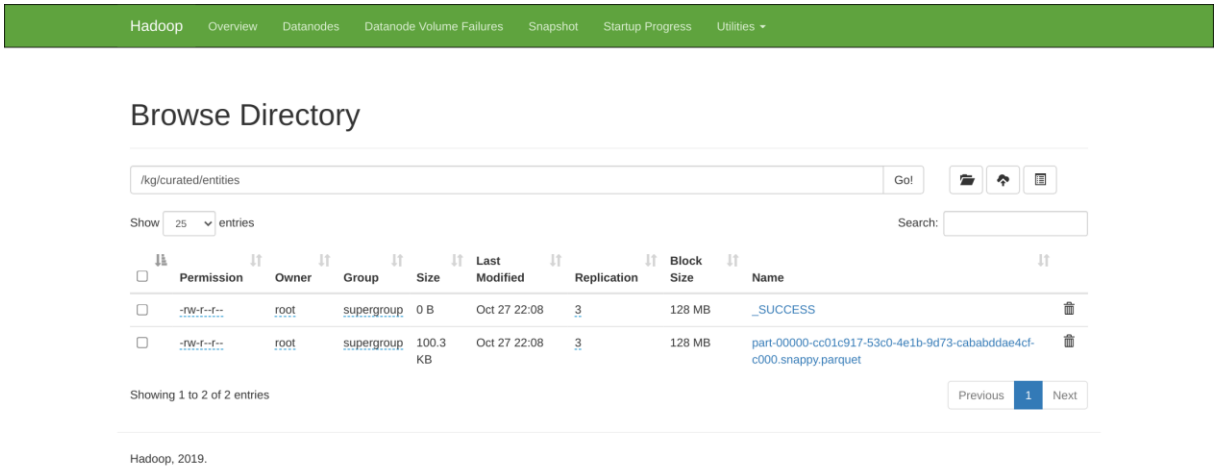


Figure 5 – Zone curée HDFS : entités au format Parquet produites par Spark

6. Chargement dans Neo4j

Le chargeur (pipeline/04_load_neo4j.py) relit les Parquet avec Spark puis écrit dans Neo4j par lots (UNWIND) :

- nœuds :Entity avec un label secondaire par type (:Company, :Person…), alias, description, documents sources ;
- nœuds :Document et relations MENTIONED_IN (provenance au niveau de l'entité) ;
- relations typées portant weight, sources (identifiants de documents) et evidence (phrases de preuve) ;
- contraintes d'unicité, index full-text sur noms et alias, **index vectoriel** (384 dimensions, cosinus) sur les embeddings des entités.

Graphe final : **1532 nœuds** (1473 entités + 59 documents) et **3163 relations** (dont 1324 relations sémantiques), chargé en 38 s embeddings compris.

Tableau 7 – Répartition des entités

Type d'entité	Nombre
Company	483
Product	412
Person	290
Location	104
Organization	103
Technology	81

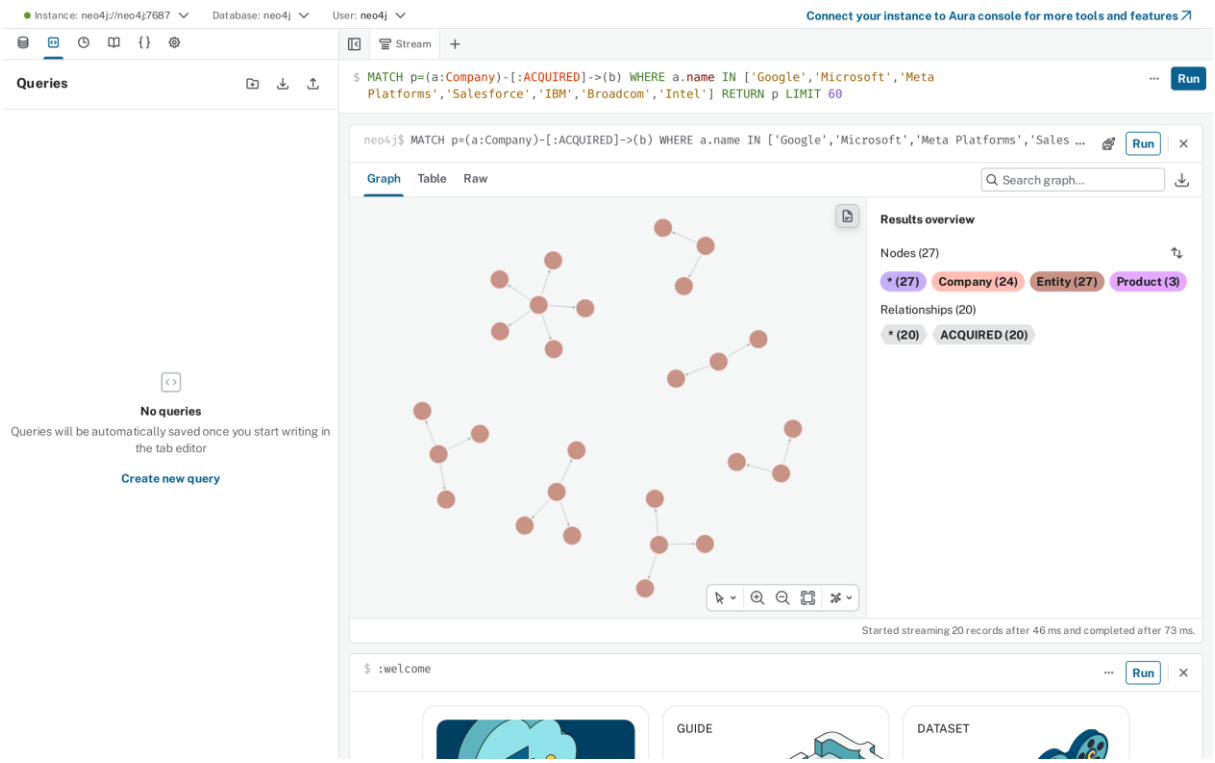


Figure 6 – Neo4j Browser : acquisitions des grands groupes technologiques

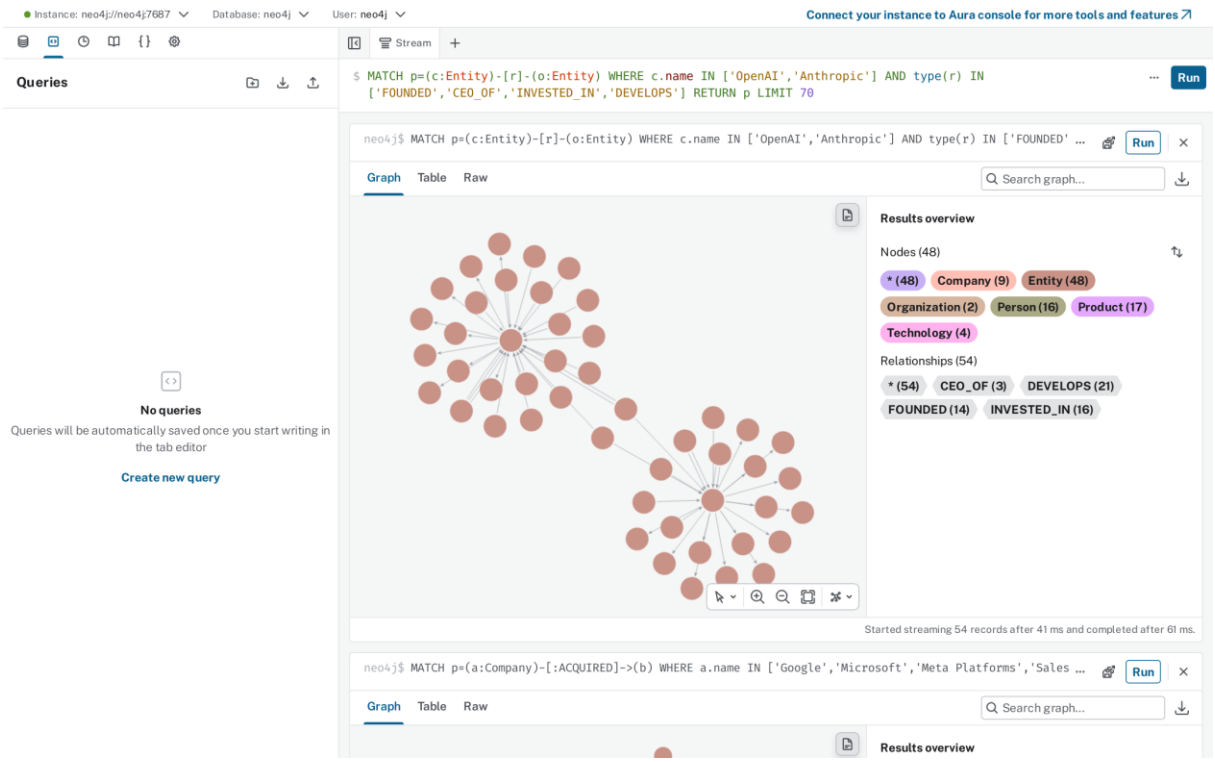


Figure 7 – Neo4j Browser : voisinage d'OpenAI et d'Anthropic (fondateurs, investisseurs, produits)

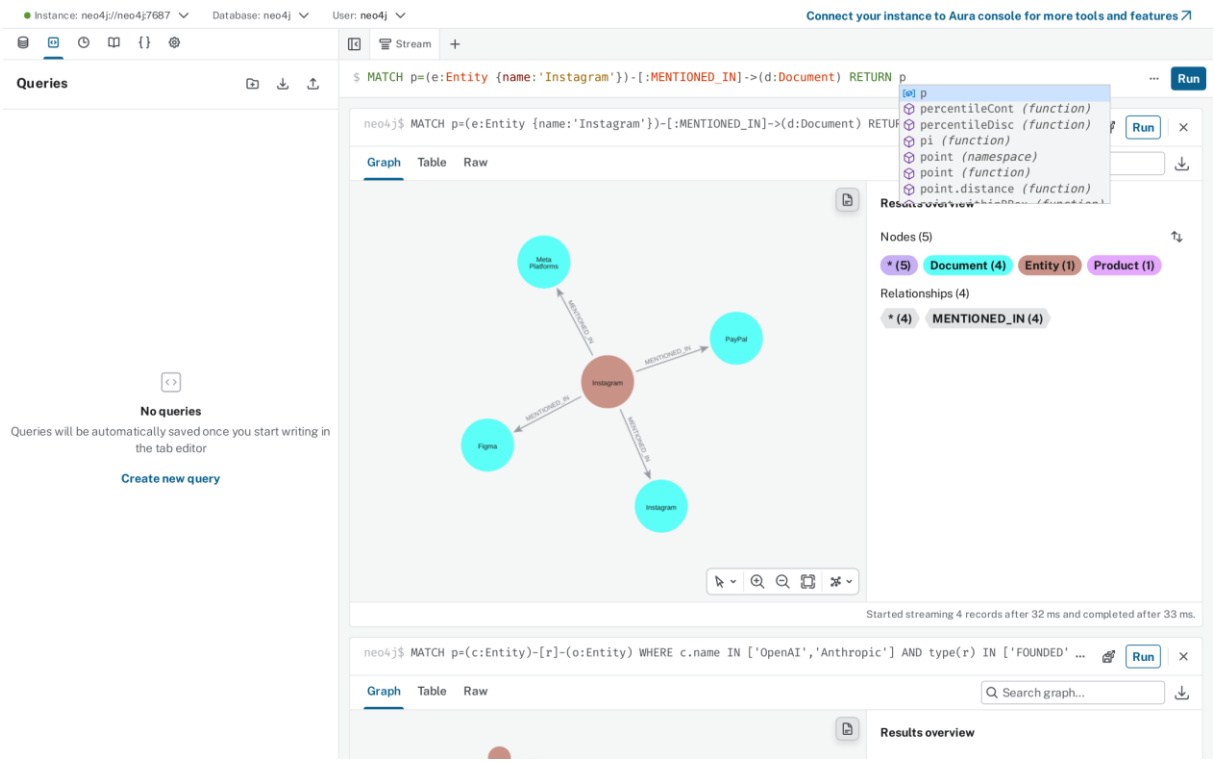


Figure 8 – Provenance : une entité reliée aux documents qui la mentionnent

7. Phase 4 — Graph Analytics (Cypher et Graph Data Science)

Le graphe sémantique est projeté en mémoire GDS (1473 nœuds, 2648 relations, non orienté) puis cinq algorithmes y sont exécutés et leurs résultats réécrits comme propriétés des nœuds :

Tableau 8 – Algorithmes GDS exécutés

Algorithme	Objectif	Résultat	Durée
PageRank	Influence d'une entité	1473 scores	0.369 s
Louvain	Détection de communautés	406 communautés, modularité 0.857	0.661 s
Betweenness	Entités « pont »	scores de centralité	0.523 s
Degree	Connectivité	degré de chaque nœud	0.185 s
WCC	Composantes connexes	385 composantes	0.133 s

Tableau 9 – Top 10 PageRank

Rang	Entité	Type	PageRank
1	Google	Company	24.765
2	Microsoft	Company	21.689
3	Apple	Company	16.288
4	IBM	Company	15.484
5	OpenAI	Company	15.193
6	Anthropic	Company	15.164
7	Mistral AI	Company	14.261
8	Red Hat	Company	13.792
9	Uber	Company	13.412
10	Nvidia	Company	12.884

Tableau 10 – Plus grandes communautés Louvain

Taille	Principaux membres
97	Tesla, SpaceX, SpaceX Super Intelligence, Neuralink, Elon Musk, X Corp., Twitter, X.com
83	Google, Alphabet Inc., Syed Anas Mohiuddin, Larry Page, Sergey Brin, PageRank, Granite Systems, Andy Bechtolsheim
72	IBM, Red Hat, Cygnus Solutions, Bob Young, Project Lightwell, MSN, AOL, Arthur L. Samuel
70	Figma, Slack, GitHub, Instagram, Andreessen Horowitz, Atlassian, Kleiner Perkins, Mike Krieger
66	OpenAI, Anthropic, San Francisco, California, OpenAI Global, LLC, OpenAI LP, John Schulman, Sam Altman

La modularité élevée (0.86) montre une structure communautaire nette et interprétable : un écosystème Elon Musk (Tesla, SpaceX, Neuralink, xAI, X/Twitter), un pôle Google/Alphabet, un pôle IBM/Red Hat, un pôle IA générative (OpenAI, Anthropic, Sam Altman) et un pôle start-ups/capital-risque (Instagram, Slack, Figma, Andreessen Horowitz). Les entités de plus forte betweenness (Microsoft, Anthropic, Google) sont celles qui relient ces communautés via des investissements et partenariats.

Exemple de requête de chemin : le plus court chemin entre Elon Musk et Microsoft est Elon Musk → OpenAI → Microsoft (relations INVESTED_IN, INVESTED_IN).

Exemple de requête multi-hop écrite à la main :

```
MATCH (g:Company)-[:ACQUIRED|OWNS]->(c)<-[:FOUNDED]-(p:Person)
        WHERE toLower(g.name) CONTAINS 'google' OR toLower(g.name) CONTAINS 'alphabet'
        RETURN g.name AS acquereur, c.name AS entreprise, collect(p.name) AS fondateurs
```

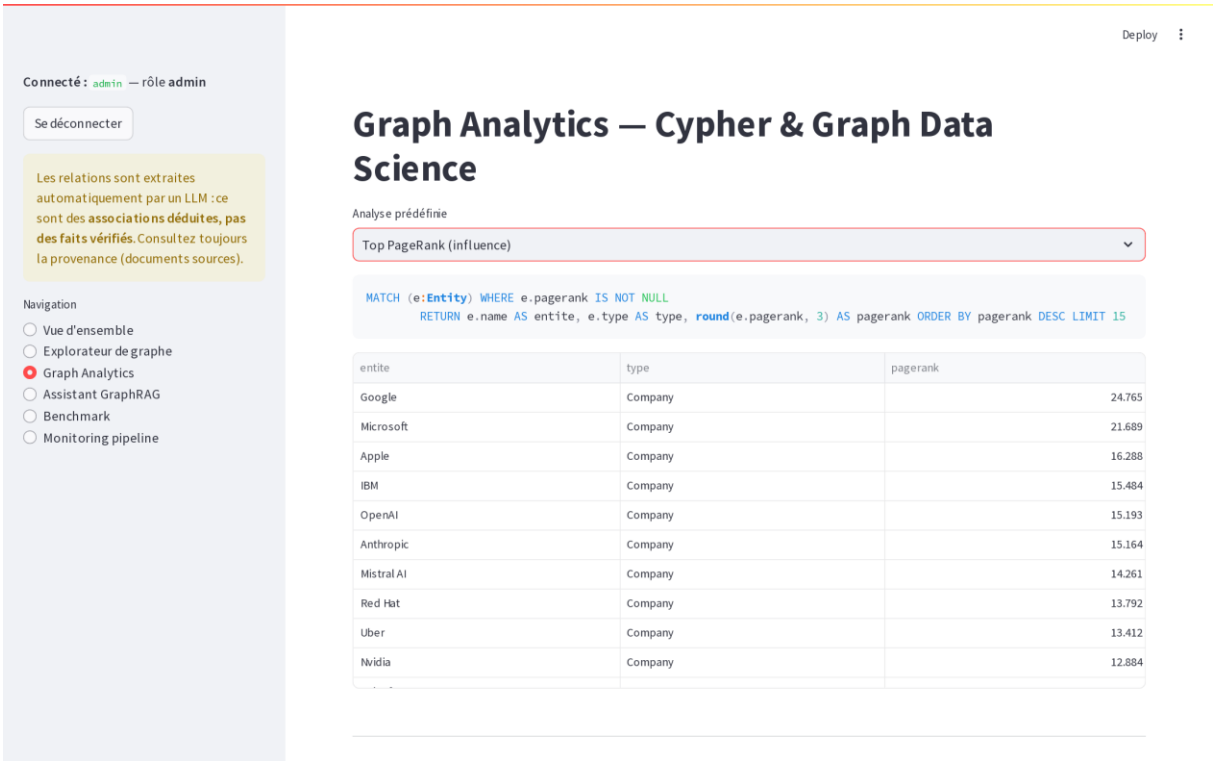


Figure 9 – Module Graph Analytics : classement PageRank

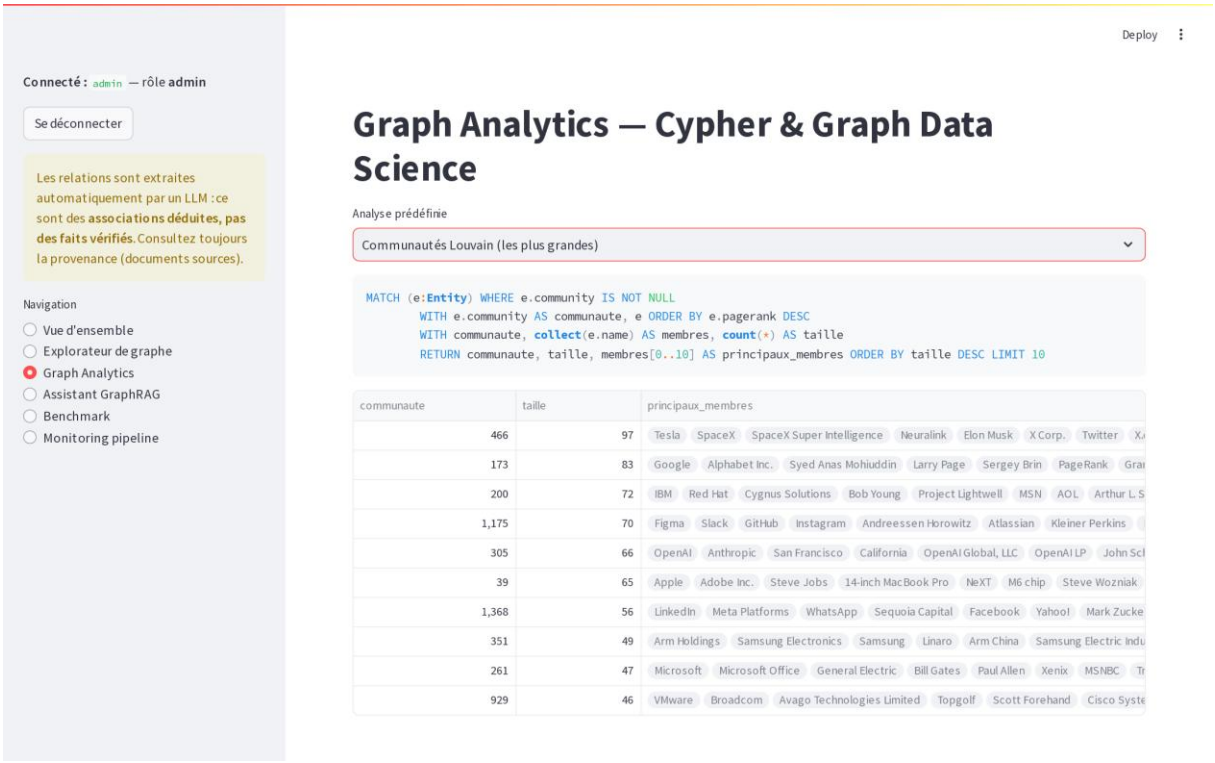


Figure 10 – Module Graph Analytics : communautés Louvain

8. Phase 5 — GraphRAG et Text2Cypher

8.1 Pipeline GraphRAG

1. **Liaison d'entités hybride** : le LLM liste les entités citées dans la question, retrouvées par l'index full-text (recherche exacte + floue) ; en parallèle, la question est vectorisée et l'index vectoriel renvoie les 5 entités les plus proches (score > 0,6).
2. **Récupération du sous-graphe** : voisinage à 1–2 sauts des entités graines (hors MENTIONED_IN), classé par lien direct, PageRank et poids, limité à 90 triplets.
3. **Génération** : les triplets sont sérialisés avec leurs documents sources et phrases de preuve ; le LLM répond uniquement à partir de ces faits, cite les documents et indique un niveau de confiance. S'il manque de faits, il doit le dire au lieu d'inventer.

```
(Google:Company) -[ACQUIRED]-> (Google DeepMind:Company) | docs: wiki_002,wiki_016 | evidence: "..."
```

8.2 Text2Cypher

Le LLM reçoit le schéma du graphe (labels, types et sens des relations) et génère une requête Cypher exécutée directement. Les requêtes contenant des clauses d'écriture (CREATE, MERGE, DELETE, SET…) sont rejetées.

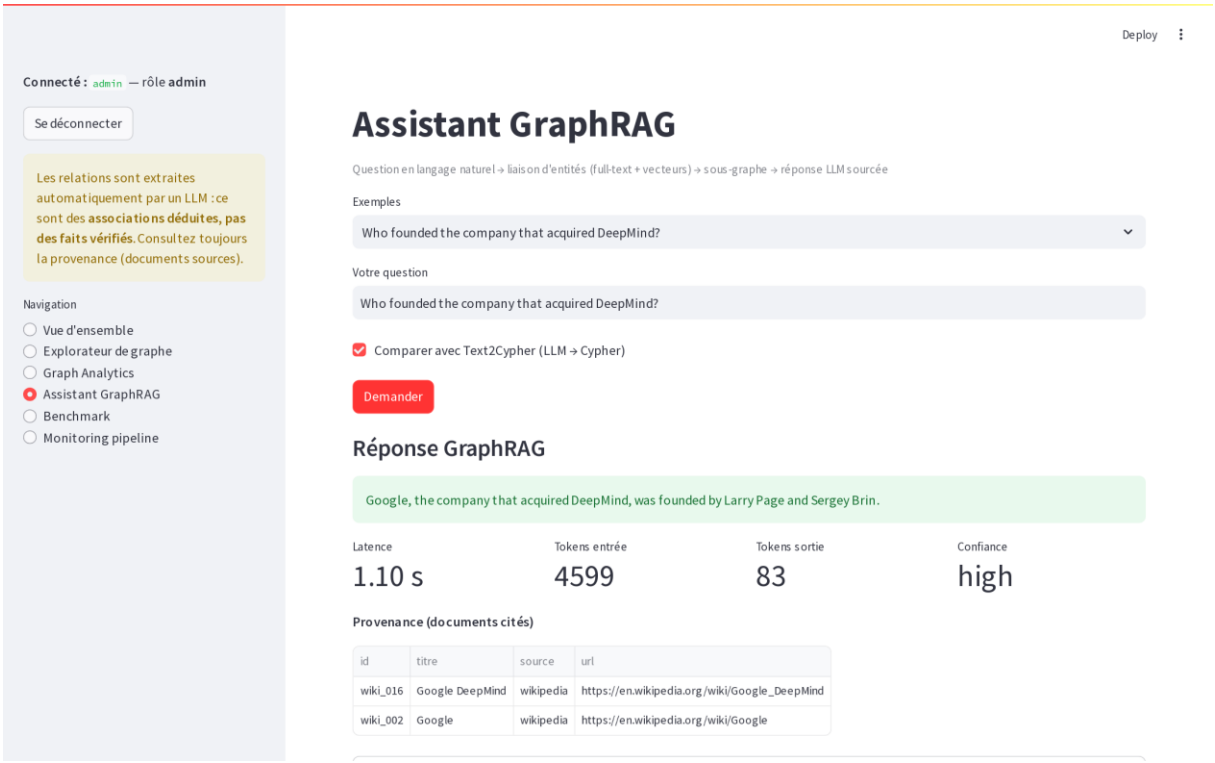


Figure 11 – Assistant GraphRAG : réponse multi-hop correcte avec documents cités

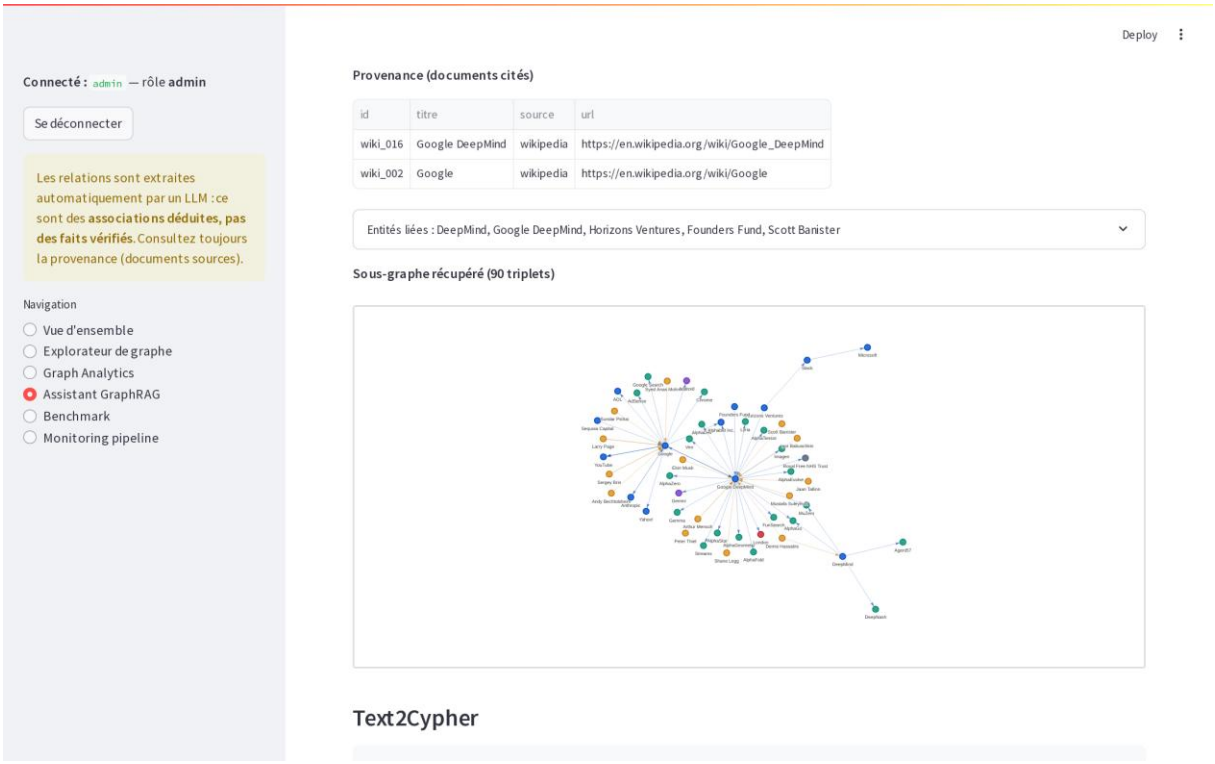


Figure 12 – Sous-graphe récupéré et entités liées

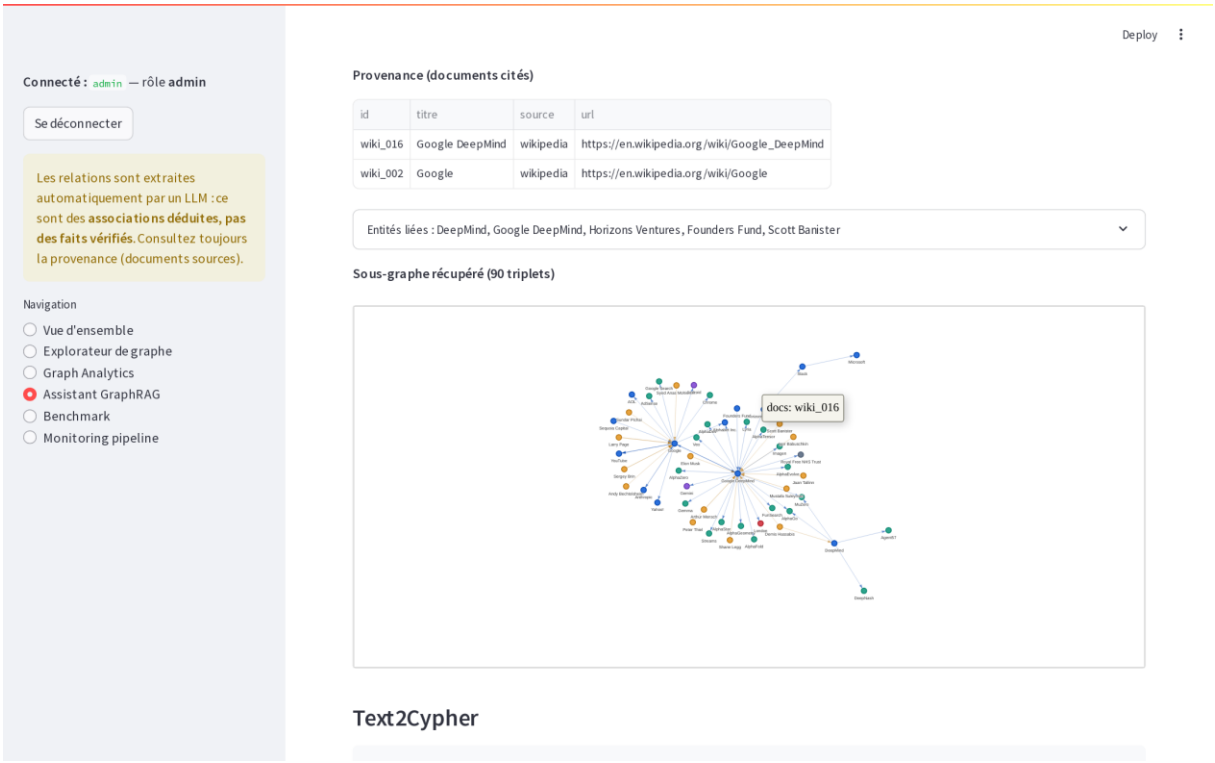


Figure 13 – Comparaison avec Text2Cypher : requête générée et résultat

9. Phase 6 — Évaluation

9.1 Qualité de l'extraction

Rappel : les faits Wikidata (fondateurs, PDG, siège, organisation mère, propriétaire) des 40 entreprises sont recherchés dans le graphe, avec un rapprochement flou des noms. Comme les articles sont tronqués, on distingue le rappel sur tous les faits et le rappel sur les faits dont l'objet apparaît réellement dans le texte fourni au LLM. **Précision** : un échantillon aléatoire de 60 relations est soumis à un juge LLM qui vérifie si la phrase de preuve soutient le triplet (sens compris).

Tableau 11 – Rappel de l'extraction mesuré contre Wikidata

Relation Wikidata	Faits	Rappel (tous)	Faits présents dans le texte	Rappel (dans le texte)
FOUNDED	98	59 %	86	66 %
HEADQUARTERED_IN	45	62 %	37	76 %
CEO_OF	33	48 %	20	75 %
SUBSIDIARY_OF	8	75 %	8	75 %
OWNED_BY	55	14 %	19	32 %
Total	239	48 %	170	66 %

Tableau 12 – Précision sur un échantillon de 60 relations (juge LLM)

Relation	Précision jugée
PARTNERS_WITH	100 %
RELATED_TO	100 %
CEO_OF	100 %
SUBSIDIARY_OF	100 %
HEADQUARTERED_IN	80 %
FOUNDED	73 %

INVESTED_IN	67 %
DEVELOPS	67 %
ACQUIRED	67 %
OWNS	50 %
WORKS_FOR	33 %
Global	72 %

Le LLM retrouve **66 % des faits présents dans le texte** avec une **précision d'environ 72 %**. Les relations factuelles et explicites (CEO_OF, SUBSIDIARY_OF, HEADQUARTERED_IN) sont fiables ; les relations plus interprétatives (WORKS_FOR, OWNS, INVESTED_IN) génèrent davantage d'erreurs de sens ou de sur-interprétation. Le rappel faible sur OWNED_BY s'explique par des actionnaires (fonds, sociétés holding) que Wikidata recense mais que les textes mentionnent rarement.

9.2 Benchmark comparatif

Un jeu de référence de **21 questions** a été construit à partir de faits publics (et non à partir du graphe), réparti en quatre catégories : un saut (7), agrégation (3), multi-hop (8) et sémantique (3). Pour chaque question, un expert a écrit la requête Cypher correspondante. Le **rappel** est la part des éléments attendus présents dans la réponse ; une question est « exacte » si tous les éléments sont trouvés.

Tableau 13 – Résultats globaux sur 21 questions

Approche	Rappel moyen	Questions exactes	Latence médiane	Latence moyenne	Tokens (entrée+sortie)	Coût total
Cypher expert	91 %	81 %	0.05 s	0.06 s	0	0.0000 $
Text2Cypher	82 %	71 %	1.02 s	2.83 s	10 908	0.0015 $
GraphRAG	86 %	76 %	2.01 s	5.06 s	97 628	0.0103 $

Tableau 14 – Rappel moyen par catégorie de question

Catégorie	Cypher expert	Text2Cypher	GraphRAG
Un saut	100 %	100 %	100 %
Multi-hop	83 %	71 %	75 %
Agrégation	89 %	89 %	89 %
Sémantique	93 %	60 %	80 %

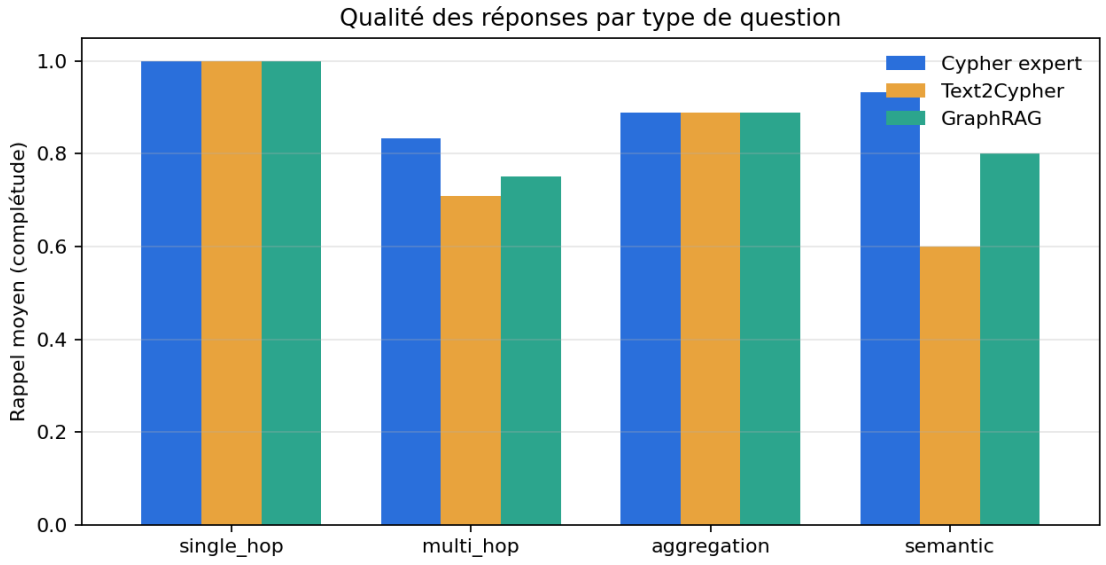


Figure 14 – Rappel par catégorie et par approche

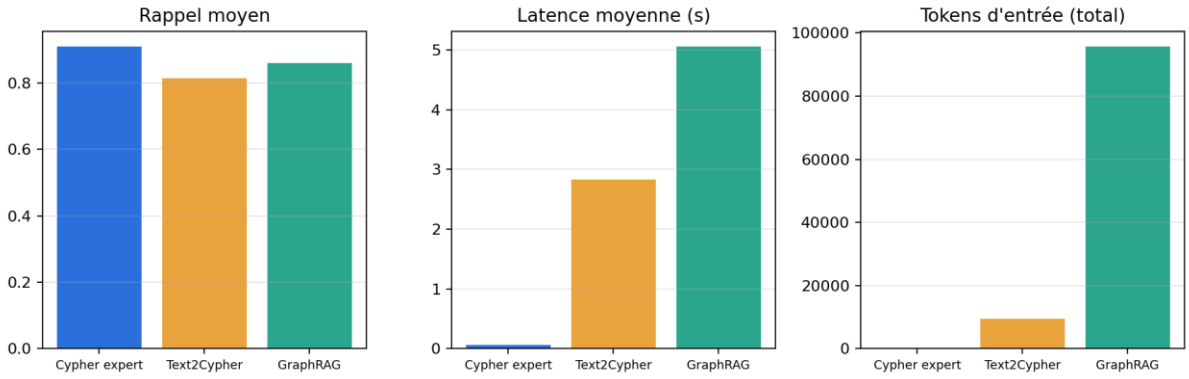


Figure 15 – Compromis qualité / latence / consommation de tokens

Tableau 15 – Rappel par question (T2C = Text2Cypher)

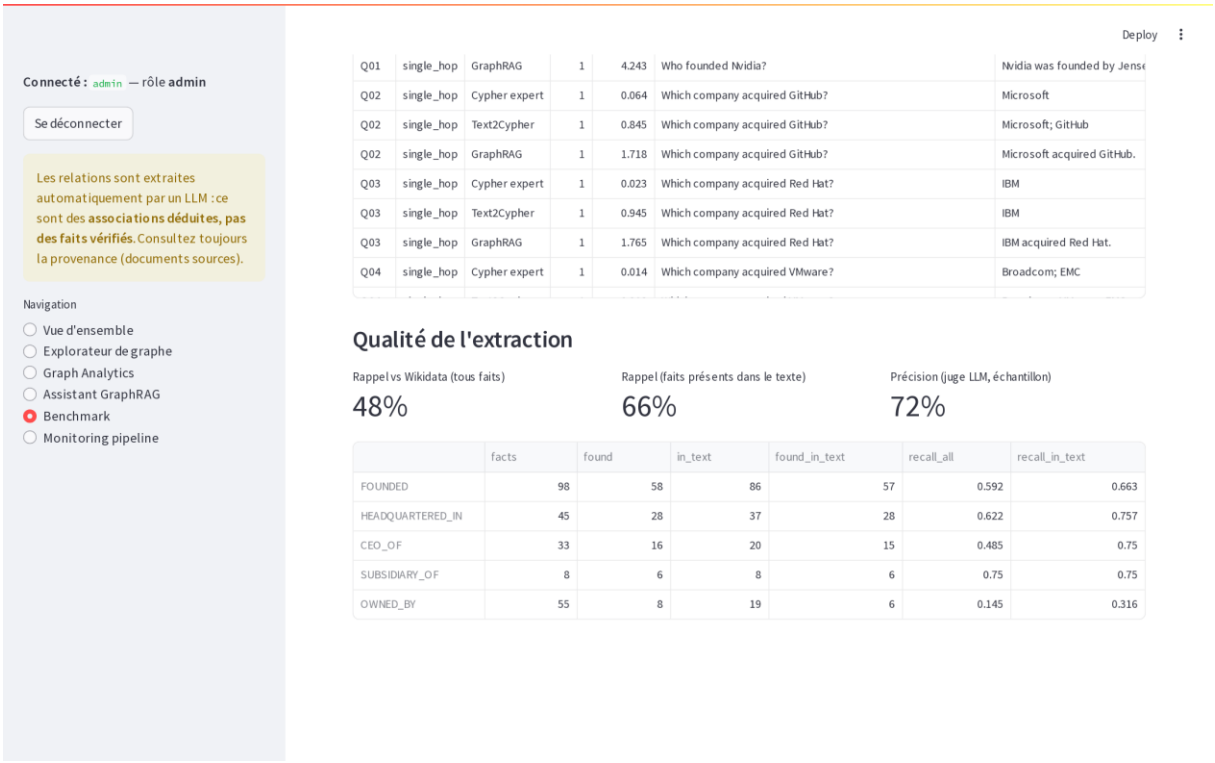
ID	Question	Cypher	T2C	GraphRAG
Q01	Who founded Nvidia?	1.00	1.00	1.00
Q02	Which company acquired GitHub?	1.00	1.00	1.00
Q03	Which company acquired Red Hat?	1.00	1.00	1.00
Q04	Which company acquired VMware?	1.00	1.00	1.00
Q05	Which company acquired Mobileye?	1.00	1.00	1.00
Q06	Who is the CEO of Nvidia?	1.00	1.00	1.00
Q07	Where is Apple headquartered?	1.00	1.00	1.00
Q08	Which companies has Microsoft acquired?	1.00	1.00	1.00
Q09	Which companies did Elon Musk found or co-found?	1.00	1.00	1.00
Q10	Which companies has Google acquired?	0.67	0.67	0.67
Q11	Who founded the company that acquired Instagram?	0.00	0.00	0.00
Q12	Where is the company that acquired Red Hat headquartered?	1.00	1.00	1.00
Q13	Who is the CEO of the company that develops ChatGPT?	1.00	1.00	1.00
Q14	Who founded the company that acquired DeepMind?	1.00	1.00	1.00
Q15	Who founded the company that develops Claude, and which company did they work for before?	0.67	0.67	1.00
Q16	Which companies invested in the company that develops Claude?	1.00	1.00	1.00
Q17	Who is the CEO of the company that owns WhatsApp?	1.00	0.00	1.00
Q18	Who founded the company that acquired Slack?	1.00	1.00	0.00
Q19	Which companies design semiconductor chips or GPUs?	0.80	0.20	0.80
Q20	Which companies build large language models or AI chatbots?	1.00	0.60	0.60
Q21	Which companies are involved in electric vehicles or spaceflight?	1.00	1.00	1.00

9.3 Analyse des erreurs

- **Q11 (fondateurs de l'acquéreur d'Instagram) — échec des trois approches.** Le graphe contient deux nœuds distincts « Facebook » et « Meta Platforms » : la résolution d'entités par similarité de noms ne peut pas détecter un changement de raison sociale. De plus, la relation Mark Zuckerberg -[FOUNDED]-> Facebook n'a pas été extraite. GraphRAG l'indique honnêtement (« les fondateurs ne figurent pas dans les faits ») au lieu d'halluciner : c'est le comportement souhaité.
- **Erreur d'extraction propagée.** Le graphe contient la relation erronée Elon Musk -[FOUNDED]-> Anthropic, issue d'une sur-interprétation du LLM. Text2Cypher et Cypher la renvoient telle quelle (Q15) ; GraphRAG, qui dispose aussi des phrases de preuve, ne l'a pas reprise dans sa réponse.
- **Questions sémantiques.** La requête Cypher experte par mots-clés obtient un bon rappel mais renvoie beaucoup de bruit (ex. Boeing, Netscape pour « véhicules électriques ou spatial »). Text2Cypher choisit

des mots-clés trop étroits (rappel de 20 % sur Q19). GraphRAG combine recherche vectorielle et raisonnement et donne des réponses plus pertinentes, sans être exhaustif.

- **Text2Cypher** échoue surtout par un mauvais sens de relation ou un nom trop spécifique dans le filtre ; une requête a pris près de 20 s à cause des réessais de l'API (quota du palier gratuit).



Q01	single_hop	GraphRAG	1	4.243	Who founded Nvidia?	Nvidia was founded by Jens
Q02	single_hop	Cypher expert	1	0.064	Which company acquired GitHub?	Microsoft
Q02	single_hop	Text2Cypher	1	0.845	Which company acquired GitHub?	Microsoft; GitHub
Q02	single_hop	GraphRAG	1	1.718	Which company acquired GitHub?	Microsoft acquired GitHub.
Q03	single_hop	Cypher expert	1	0.023	Which company acquired Red Hat?	IBM
Q03	single_hop	Text2Cypher	1	0.945	Which company acquired Red Hat?	IBM
Q03	single_hop	GraphRAG	1	1.765	Which company acquired Red Hat?	IBM acquired Red Hat.
Q04	single_hop	Cypher expert	1	0.014	Which company acquired VMware?	Broadcom; EMC

	facts	found	in_text	found_in_text	recall_all	recall_in_text
FOUNDED	98	58	86	57	0.592	0.663
HEADQUARTERED_IN	45	28	37	28	0.622	0.757
CEO_OF	33	16	20	15	0.485	0.75
SUBSIDIARY_OF	8	6	8	6	0.75	0.75
OWNED_BY	55	8	19	6	0.145	0.316

Figure 16 – Module Benchmark de l'application

10. Phase 7 — Application, API, monitoring et contrôle d'accès

L'interface **Streamlit** regroupe : vue d'ensemble du graphe, explorateur interactif (pyvis), catalogue d'analyses Cypher/GDS et console Cypher en lecture seule, assistant GraphRAG avec provenance, résultats du benchmark et monitoring. Une **API REST FastAPI** expose les mêmes services (/ask, /text2cypher, /analytics, /cypher, /health).

Tableau 16 – Contrôle d'accès par rôles (interface : mot de passe haché SHA-256 ; API : en-tête X-API-Key)

Rôle	Accès
viewer	tableaux de bord, explorateur, assistant GraphRAG, benchmark
analyst	+ console Cypher libre (lecture seule) et endpoint /cypher de l'API
admin	+ monitoring du pipeline (journal des exécutions, fichiers HDFS, état Neo4j)

Le **monitoring** repose sur un journal d'exécution (pipeline_runs.jsonl) alimenté par chaque étape : volumes, erreurs, tokens, coût estimé, durée. Les requêtes Cypher saisies par les utilisateurs sont filtrées pour interdire toute écriture.



Figure 17 – Authentification

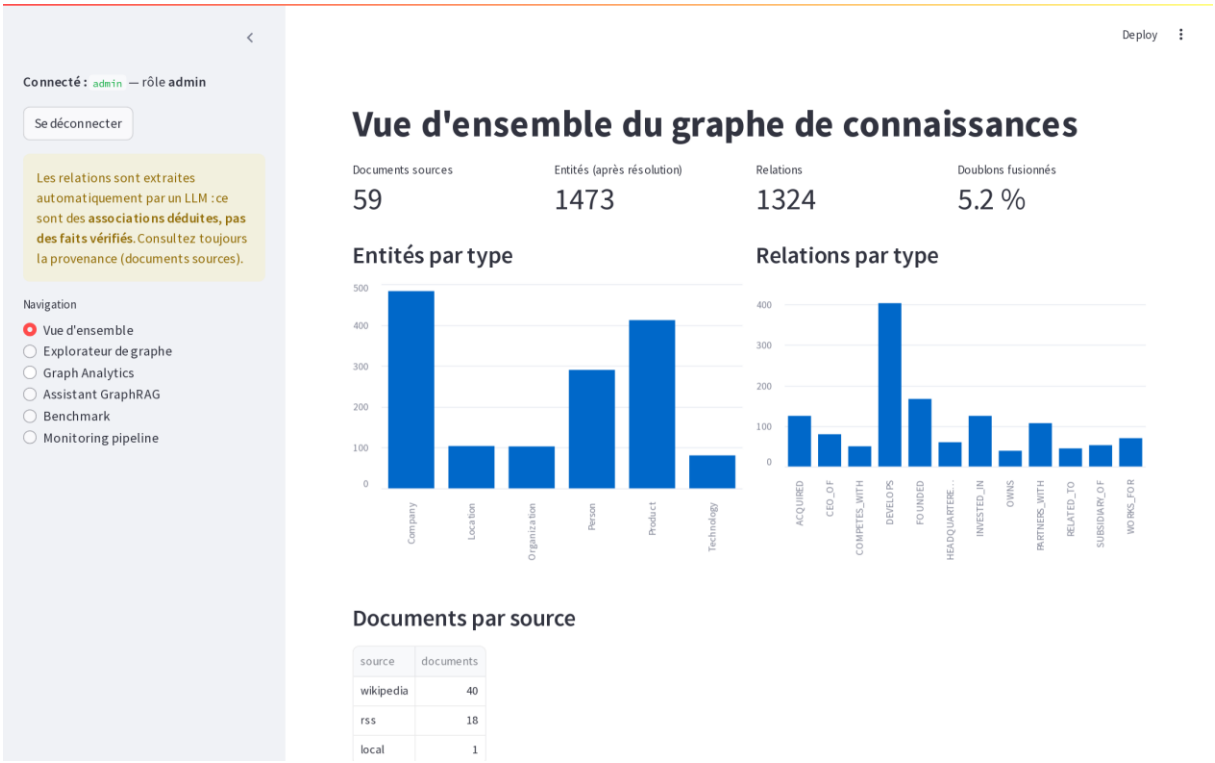


Figure 18 – Vue d'ensemble du graphe de connaissances

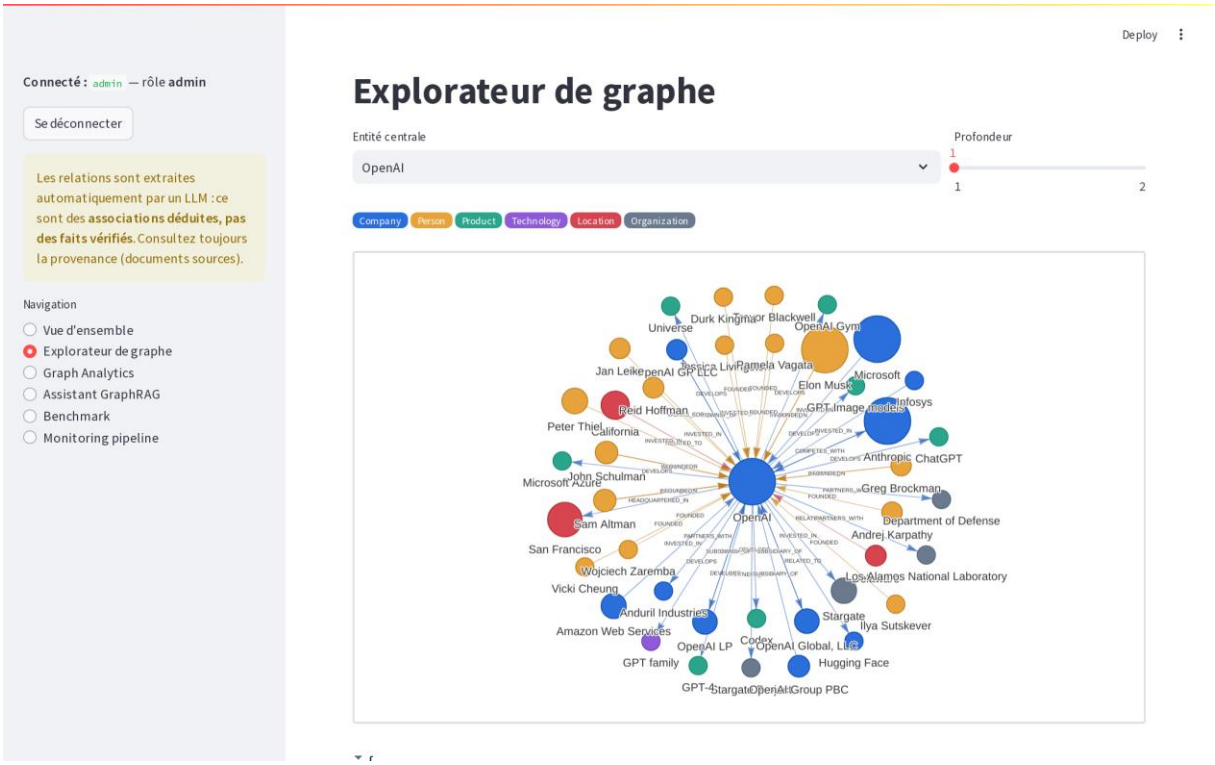
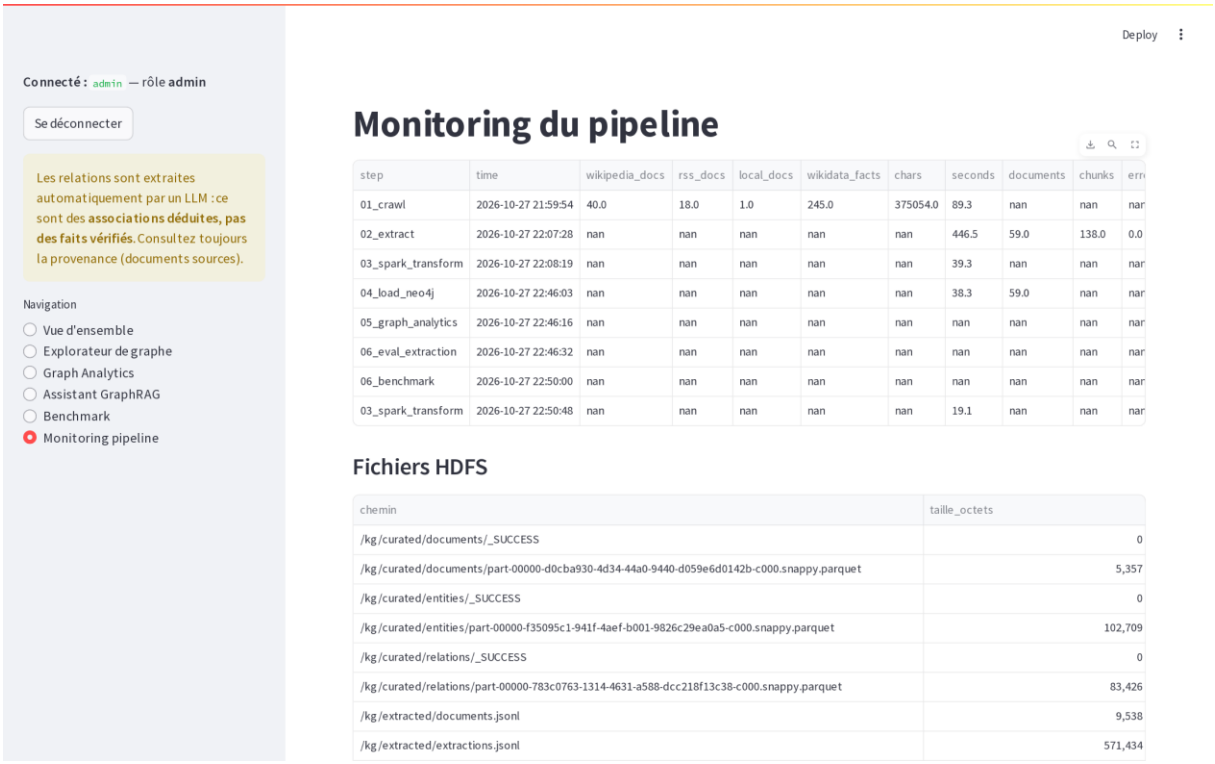


Figure 19 – Explorateur de graphe : voisinage d'OpenAI coloré par type d'entité



step	time	wikipedia_docs	rss_docs	local_docs	wikidata_facts	chars	seconds	documents	chunks	err
01_crawl	2026-10-27 21:59:54	40.0	18.0	1.0	245.0	375054.0	89.3	nan	nan	nan
02_extract	2026-10-27 22:07:28	nan	nan	nan	nan	nan	446.5	59.0	138.0	0.0
03_spark_transform	2026-10-27 22:08:19	nan	nan	nan	nan	nan	39.3	nan	nan	nan
04_load_neo4j	2026-10-27 22:46:03	nan	nan	nan	nan	nan	38.3	59.0	nan	nan
05_graph_analytics	2026-10-27 22:46:16	nan	nan	nan	nan	nan	nan	nan	nan	nan
06_eval_extraction	2026-10-27 22:46:32	nan	nan	nan	nan	nan	nan	nan	nan	nan
06_benchmark	2026-10-27 22:50:00	nan	nan	nan	nan	nan	nan	nan	nan	nan
03_spark_transform	2026-10-27 22:50:48	nan	nan	nan	nan	nan	19.1	nan	nan	nan

chemin	taille_octets
/kg/curated/documents/_SUCCESS	0
/kg/curated/documents/part-00000-d0cba930-4d34-44a0-9440-d059e6d0142b-c000.snappy.parquet	5,357
/kg/curated/entities/_SUCCESS	0
/kg/curated/entities/part-00000-f35095c1-941f-4aef-b001-9826c29ea0a5-c000.snappy.parquet	102,709
/kg/curated/relations/_SUCCESS	0
/kg/curated/relations/part-00000-783c0763-1314-4631-a588-dcc218f13c38-c000.snappy.parquet	83,426
/kg/extracted/documents.jsonl	9,538
/kg/extracted/extractions.jsonl	571,434

Figure 20 – Monitoring : journal des exécutions du pipeline (rôle admin)

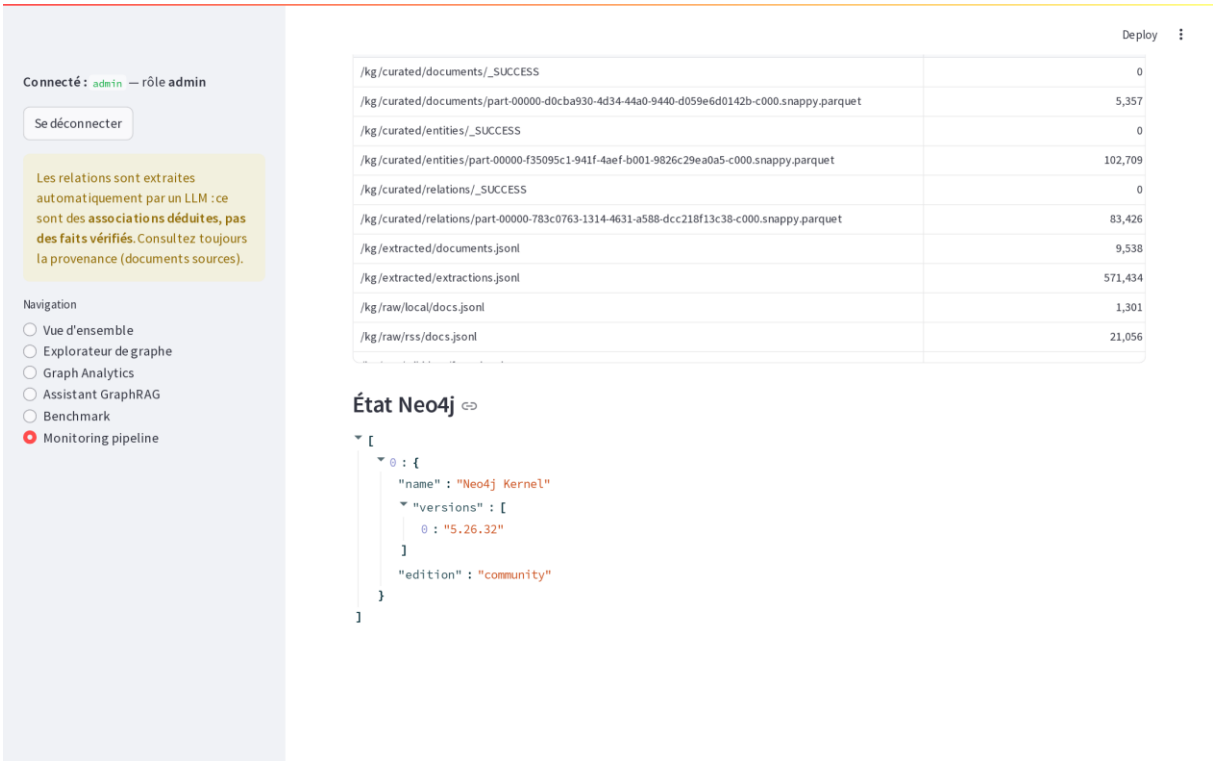


Figure 21 – Monitoring : fichiers HDFS et état de Neo4j

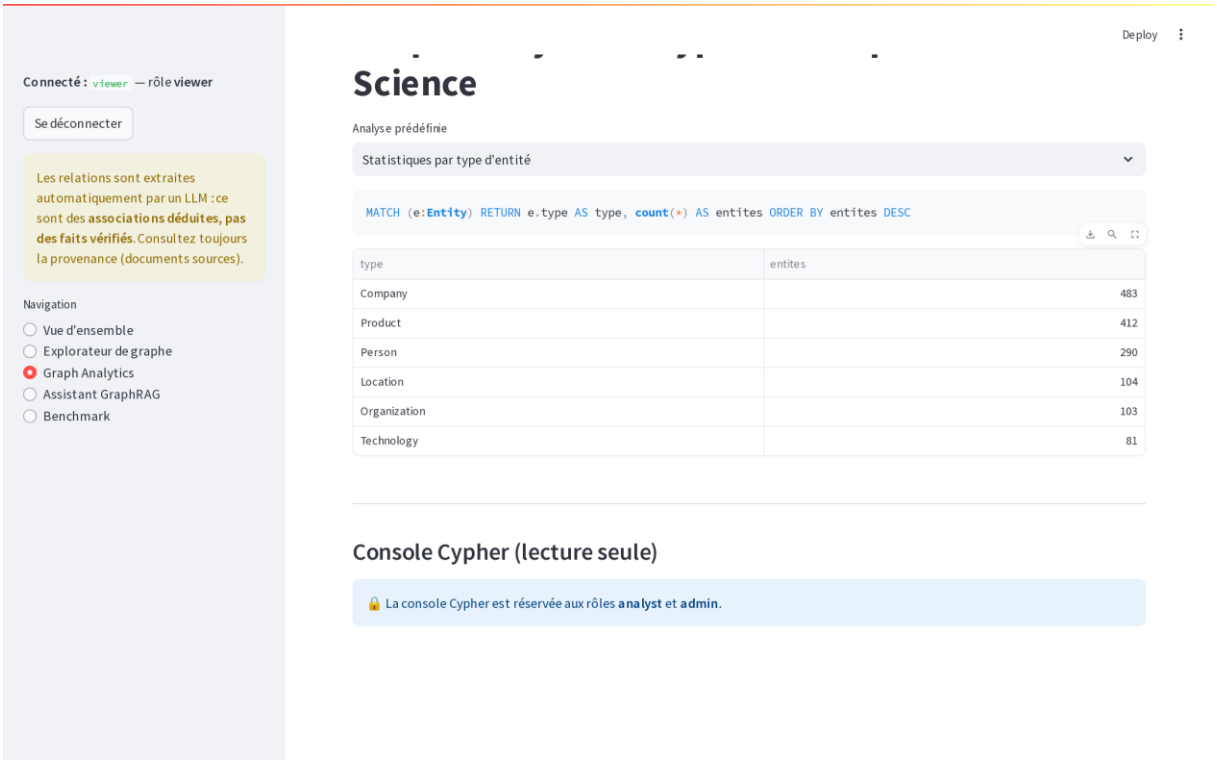


Figure 22 – Contrôle d'accès : la console Cypher est verrouillée pour le rôle viewer

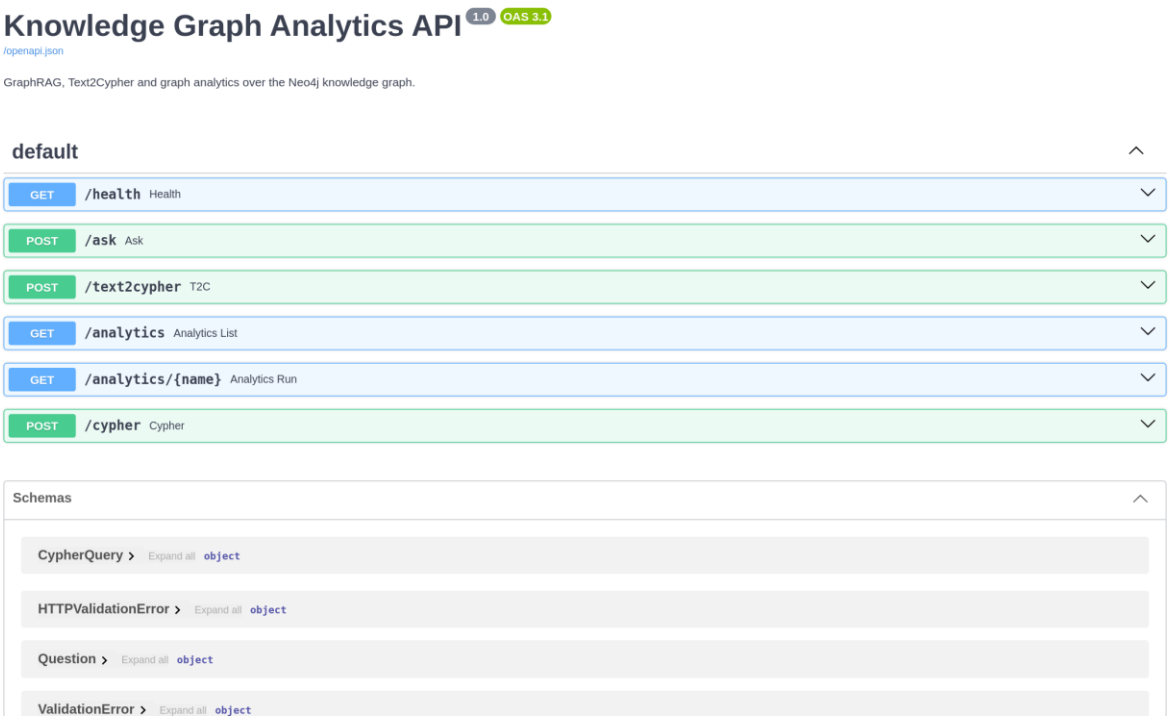


Figure 23 – Documentation Swagger de l'API REST

11. Réponses aux questions de recherche

Q1 — Précision de l'extraction par LLM. Sur les faits effectivement présents dans les textes, le LLM en retrouve 66 %, avec une précision jugée d'environ 72 %. Un modèle léger et bon marché suffit pour les relations explicites, mais les relations interprétatives exigent une validation (juge LLM, règles de cohérence de types ou vérification humaine).

Q2 — Impact de la résolution d'entités. La résolution réduit de 5.2 % le nombre d'entités et corrige 32 conflits de type. Sans elle, « Nvidia », « NVIDIA » et « Nvidia Corporation » seraient trois nœuds et les requêtes multi-hop se briseraient. Le cas Facebook / Meta Platforms montre sa limite : la similarité de noms ne capte pas les changements de nom ; il faudrait lier les entités à un identifiant externe (Wikidata QID).

Q3 — GraphRAG vs Cypher direct. Quand un expert écrit la requête, Cypher reste le plus complet (91 % de rappel) et de loin le plus rapide (~51 ms). GraphRAG (86 %) le talonne sans aucune connaissance de Cypher ni du schéma et dépasse Text2Cypher (82 %), surtout sur les questions multi-hop et sémantiques, où la recherche vectorielle compense le vocabulaire absent du schéma.

Q4 — Compromis qualité / complétude / latence / coût. Cypher : coût nul, millisecondes, mais nécessite un expert. Text2Cypher : 1 appel LLM, ~1.0 s, très peu de tokens, mais fragile. GraphRAG : 2 appels LLM, ~2.0 s en médiane, environ 4560 tokens par question (10× Text2Cypher) à cause du contexte de triplets, mais réponses rédigées, sourcées et robustes. GraphRAG est moins exhaustif sur les listes longues car le contexte est tronqué à 90 triplets.

Q5 — Apport de la provenance. Chaque relation conserve ses documents sources et une phrase de preuve. GraphRAG cite les documents utilisés, l'utilisateur peut vérifier la réponse, et l'évaluateur LLM a pu mesurer

la précision grâce à ces preuves. Les relations confirmées par plusieurs documents (poids ≥ 2) sont les plus fiables et peuvent être privilégiées.

12. Éthique, conformité et gouvernance des données

- Collecte limitée à des **sources publiques et licites** (Wikipédia, Wikidata, flux RSS publics), dans le respect des politiques d'accès (User-Agent, limitation du débit).
- Les personnes présentes dans le graphe sont des personnalités publiques citées dans des sources encyclopédiques.
- Les relations sont **déduites automatiquement** : l'interface affiche un avertissement permanent indiquant qu'il ne s'agit pas de faits vérifiés, et chaque réponse est accompagnée de sa provenance.
- **Contrôle d'accès** par rôles et par clés d'API ; console Cypher en lecture seule.
- **Rétention** : les données brutes sont versionnées dans HDFS par zone (raw / extracted / curated) et peuvent être purgées par source ; aucune donnée personnelle sensible n'est collectée.
- Secrets (clé d'API) stockés dans un fichier .env exclu du contrôle de version.

13. Limites et perspectives

- **Échelle** : 59 documents, Spark en mode local. L'architecture est prête pour un cluster (Spark standalone/YARN, plusieurs DataNodes) ; l'extraction LLM devient alors le goulot d'étranglement (quota d'API, coût).
- **Résolution d'entités** : ajouter un ancrage sur les identifiants Wikidata et un rapprochement par embeddings pour traiter les renommages (Facebook → Meta) ; quelques fausses fusions (IBM 704 / IBM 7094) suggèrent un seuil plus strict pour les produits.
- **Évaluation** : 21 questions et un juge LLM du même fournisseur que l'extracteur (biais possible) ; une annotation humaine d'un échantillon renforcerait les conclusions.
- **GraphRAG** : résumés de communautés (GraphRAG « global » de Microsoft) pour les questions de synthèse, re-classement des triplets et routage automatique vers Cypher quand la question est structurée.
- **Production** : orchestration (Airflow), métriques Prometheus/Grafana, authentification OAuth.

14. Conclusion

Le projet livre une plateforme complète et reproductible (docker compose up) qui va de la collecte de sources hétérogènes à un assistant en langage naturel, en passant par HDFS, l'extraction par LLM, Spark, Neo4j et la Graph Data Science. Le graphe obtenu (1473 entités, 1324 relations) révèle une structure communautaire claire de l'écosystème technologique. La comparaison montre que **les deux approches sont complémentaires** : Cypher est imbattable en précision, latence et coût quand la question est connue et que l'utilisateur maîtrise le schéma ; GraphRAG démocratise l'accès au graphe, gère les questions sémantiques et rend ses réponses explicables grâce à la provenance, au prix d'une latence et d'un coût d'inférence plus élevés. Dans les deux cas, la qualité finale dépend d'abord de la qualité de l'extraction et de la résolution d'entités.

Annexe A — Exécution du projet

```
docker compose up -d --build
docker compose exec app bash run_pipeline.sh      # phases 1 à 6

Interface : http://localhost:8501   (viewer/viewer123, analyst/analyst123, admin/admin123)
API       : http://localhost:8000/docs   (X-API-Key: demo-viewer-key | demo-analyst-key)
Neo4j     : http://localhost:7475   (neo4j / kgpassword123, bolt port 7688)
HDFS      : http://localhost:9870
```

Annexe B — Organisation du code

Dossier / fichier	Contenu
common/config.py	schéma du graphe, sources, paramètres
common/llm.py	client Gemini (JSON contraint, cache, réessais, tokens) + embeddings
common/entity_resolution.py	normalisation, harmonisation des types, fusion floue
pipeline/01…05_*.py	crawl → extraction → Spark → Neo4j → GDS
analytics/queries.py	catalogue de requêtes Cypher et GDS
graphrag/engine.py	GraphRAG (liaison hybride, sous-graphe, génération) et Text2Cypher
benchmark/	questions de référence, benchmark, évaluation de l'extraction
ui/, api/	interface Streamlit avec RBAC, API FastAPI
data/reports/	résultats JSON/CSV, figures, captures d'écran